# Background: How We Got the Generics We Have

(Or, how I learned to stop worrying and love erasure)
Brian Goetz
June 2020

Before we can talk about where generics are going, we first have to talk about where they are, and how they got there. This document will focus primarily on how we arrived at the generics we have now, and why, as a means of setting the groundwork for how the generics we have now will influence the "better" generics we are trying to build.

In particular, we emphasize that erasure was in fact the sensible and pragmatic choice for adding generics to Java in 2004 — and many of the forces that led us to choose translation by erasure may still be operating today.

## Erasure

Ask any developer about Java generics, and you'll likely get an angry (though often uninformed) rant about *erasure*. Erasure is probably the most broadly and deeply misunderstood concept in Java.

Erasure is not specific to Java, nor to generics; it is a ubiquitous, and often necessary, tool for translating code at one level into a lower level (such as when compiling from Java source to bytecode, or compiling C source to native code.) This is because as we move down the stack from high-level languages to intermediate representations to native code to hardware, the type abstractions offered by the lower level are almost always simpler and weaker than those at the higher level — and rightly so. (We wouldn't want to bake the semantics of virtual dispatch into the X86 instruction set, or mimic the set of Java's primitive types in its registers.) Erasure is the technique of mapping richer types at one level to less rich types at a lower level (ideally, after performing sound type checking at the higher level), and is something compilers do every day.

For example, the Java bytecode set contains instructions for moving integers values between the stack and local variable set (`iload`, `istore`), and for doing arithmetic on ints (`iadd`, `imul`, etc.) There are similar instructions for floats (`fload`, `fstore`, `fmul`, etc), longs (`lload`, `lstore`, `lmul`), doubles (`dload`, `dstore`, `dmul`), and object references (`aload`, `astore`.) But there are no such instructions for bytes, shorts, chars, or booleans — because these types are erased to ints by the compiler, and use the `int`-movement and arithmetic instructions. This is a pragmatic design tradeoff in the design of the bytecode instruction set; it reduces the complexity of the instruction set, which in turn can improve the efficiency of the runtime. Many other features of the Java language (e.g., checked exceptions, method overloading, enums, definite assignment analysis, nested classes, capture of local variables by lambdas or local classes, etc) are "language fictions" — they are checked in the Java compiler but erased away in the translation to classfiles.

Similarly, when compiling C to native code, both signed and unsigned ints are erased into general-purpose registers (there are no separate signed vs. unsigned registers), and `const` variables are stored in mutable registers and memory locations. We don't find this sort of erasure weird at all.

### Homogeneous vs. heterogeneous translations

There are two common approaches for translating generic types in languages with parameteric polymorphism — *homogeneous* and *heterogeneous* translation. In a homogeneous translation, a generic class `Foo<T>` is translated into a single artifact, such as `Foo.class` (and same for generic methods). In a heterogeneous translation, each instantiation of a generic type or method (`Foo<String>`, `Foo<Integer>`) is treated as a separate entity, and generates separate artifacts. For example, `C++` uses a heterogeneous translation: different instantiations of a template are completely different types, with different semantics and different generated code. The types

`vector<int>` and `vector<float>` are separate types. On the one hand, this is great for type safety (each instantiation can be separately type-checked after expansion) and for the quality of generated code (as each instantiation can be separately optimized). On the other hand, this means larger code footprint (since `vector<int>` and `vector<float>` have separate code), and we cannot talk about "vector of something" (as Java does through wildcards), since each instantiation is a wholly unrelated type. (As an extreme demonstration of the possible footprint costs, Scala experimented with an `@specialized` annotation that, when applied to type variables, caused the compiler to emit specialized versions for all the primitive types. This sounds cool, but results in a explosion of generated classes, where is the number of specialized type variables in a class, so one can easily generate a 100MB JAR file from a few lines of code.)

The choice between homogeneous and heterogeneous translations involves making the sorts of tradeoffs language designers make all the time. Heterogeneous translations offer more type specificity, at the cost of greater static and dynamic footprint, and less sharing at runtime — all of which have performance implications. Homogeneous translations are more amenable to abstracting over parametric families of types, such as Java's wildcards, or C#'s declaration-site variance (both of which `C++` lacks, where there is nothing in common between `vector<int>` and `vector<float>`.) For more information on translation strategies, see this influential paper.

## Erased generics in Java

Java translates generics using a homogeneous translation. Generics are type-checked at compile time, but then a generic type like `List<String>` is erased to `List` when generating bytecode, and type variables such as `<T extends Object>` are erased to the erasure of their bound (in this case, `Object`).

If we have:

```
class Box<T> {
    private T t;

    public Box(T t) { this.t = t; }

    public Box<T> copy() { return new Box<>(t); }

    public T t() { return t; }
}
```

The `javac` compiler emits a single classfile `Box.class`, which serves as the implementation for all instantiations of `Box` — including wildcards (`Box<?>`) and raw types (`Box`). Field, method, and supertype descriptors are erased; type variables are erased to their bounds, generic types are erased to their head (`List<String>` erases to `List`) as follows:

```
class Box {
    private Object t;

    public Box(Object t) { this.t = t; }

    public Box copy() { return new Box(t); }

    public Object t() { return t; }
}
```

The generic signatures are retained (in the `Signature` attribute) so that compilers can see the generic signatures when reading the classfile, but the JVM uses only the erased descriptors in linkage. This translation scheme means that at the classfile level, both the *layout* and *API* of `Box<T>` is erased. At the *use site*, the same thing happens: references to `Box<String>` are erased to `Box`, with a synthetic cast to `String` inserted at the use site.

## Why? What were the alternatives?

It is at this point where it is tempting to huff and declare that these were obviously foolish or lazy choices, or that erasure is a dirty hack. After all, why would the compiler throw away perfectly good type information?

To better understand the question, we should also ask: were we to reify that type information, what would we expect to do with it, and what are the costs associated with that? There are several different ways we could envision using reified type parameter information:

- **Reflection.** For some, "reified generics" merely means that you can ask a `List` what it is a list of, whether using language tools like `instanceof` or pattern matching on type variables, or using reflective libraries to inquire about the type parameters.

- **Layout or API specialization.** In a language with primitive types or inline classes, it might be nice to flatten the layout of a `Pair<int, int>` to hold two ints, rather than two references to boxed objects.

- **Runtime type checking.** When a client attempts to put an `Integer` in a `List<String>` (through, say, a raw `List` reference), which would cause heap pollution, it would be nice to catch this and fail at the point where the heap pollution would be caused, rather than (maybe) detecting it later when it hits a synthetic cast.

While not mutually exclusive, these three possibilities (reflection, specialization, and type checking) are in aid of different goals (programmer convenience, performance, and safety, respectively) — and have different implications and costs. While it is easy to say "we want reification", if we drill deeper, we'll find significant divisions as to which of these are most important, and their relative costs and benefits.

To understand how erasure was the sensible and pragmatic choice here, we also have to understand what the goals, priorities and constraints, and alternatives were at the time.

## Goal: Gradual migration compatibility

Java generics adopted an ambitious requirement:

> It must be possible to evolve an existing non-generic class to be generic in a binary-compatible and source-compatible manner.

This means that existing clients and subclasses of, say, `ArrayList`, could continue to recompile without change against the generified `ArrayList<T>`, and that existing classfiles would continue to link to the methods of the generified `ArrayList<T>`. Supporting this meant that clients and subclasses of generified classes could choose to generify immediately, later, or never, and could do so independently of what maintainers of other clients or subclasses chose to do.

Without this requirement, generifying a class would require a "flag day" where all clients and subclasses have to be at least recompiled, if not modified — all at once. For a core class such as `ArrayList`, this essentially requires all the Java code in the world to be recompiled at once (or be permanently relegated to remain on Java 1.4.) Since such a "flag day" across the entire Java ecosystem was out of the question, we needed a generic type system that allowed core platform classes (as well as popular third-party libraries) to be generified without requiring clients be aware of their generification. (Worse, it wouldn't have been one flag day, but many, since it is not the case that all the world's code would have been generified in a single atomic transaction.)

Another way to state this requirement is: it was not considered acceptable to orphan all the code out there that could have been generified, or make developers choose between generics and retaining the investment they've already made in existing code. By making generification a compatible operation, the investment in that code could be retained, rather than invalidated.

The aversion to "flag days" comes from an essential aspect of Java's design: Java is *separately compiled* and *dynamically linked*. Separate compilation means that every source file is compiled into one or more class files,

rather than compiling a group of sources into a single artifact. Dynamic linkage means that references between classes are linked at run time, based on symbolic information; if class `C` invokes method `void m(int x)` in `D`, then in the classfile for `C` we record the name and descriptor (`(I)V`) of the method we are invoking, and at link time we look for a method in `D` with this name and descriptor, and if a match is found, the call site is linked.

This may sound like a lot of work, but separate compilation and dynamic linkage power one of Java's biggest advantages — you can compile `C` against one version of `D` and run with a different version of `D` on the class path (as long as you don't make any *binary incompatible* changes in `D`.).

> The pervasive commitment to dynamic linkage is what allows us to simply drop a new JAR on the class path to update to a new version of a dependency, without having to recompile anything. We do this so often we don't even notice — but if this stopped working, it would indeed be noticed.

At the time generics were introduced into Java, there was already a lot of Java code in the world, and their classfiles were full of references to APIs like `java.util.ArrayList`. If we couldn't generify these APIs compatibly, then we would have to have written *new* APIs to replace them, and worse, all of the client code of the old APIs would be stuck with an untenable choice — either stay on 1.4 forever, or rewrite them to use the new APIs, simultaneously (including not only the application code, but all third-party libraries on which the application depends.) This would have devalued almost all the Java code in existence at the time.

C# made the opposite choice — to update their VM, and invalidate their existing libraries and all the user code that dependend on it. They could do this at the time because there was comparatively little C# code in the world; Java didn't have this option at the time.

One consequence of this choice, though, is that it will be an expected occurrence that a generic class will simultaneously have both generic and non-generic clients or subclasses. This is a boon to the software development process, but it has potential consequences for type safety under such mixed usage.

## Heap pollution

Erasing in this manner, and supporting interoperability between generic and non-generic clients, creates the possibility of *heap pollution* — that what is stored in the box has a runtime type that is not compatible with the compile-time type that was expected. When a client uses a `Box<String>`, casts are inserted whenever a `T` would be assigned to a `String`, so that heap pollution is detected at the point where data transitions from the world of type variables (the implementation of `Box`) to the world of concrete types. In the presence of heap pollution, these casts can fail.

Heap pollution can come from when non-generic code uses generic classes, or when we use unchecked casts or raw types to forge a reference to a variable of the wrong generic type. (When we used unchecked casts or raw types, the compiler warns us that heap pollution might result.) For example:

```
Box<String> bs = new Box<>("hi!");   // safe
Box<?> bq = bs;                      // safe, via subtyping
Box<Integer> bi = (Box<Integer>) bq; // unchecked cast -- warning issued
Integer i = bi.get();                // ClassCastException in synthetic cast to Integer
```

The sin in this code is the unchecked cast from `Box<?>` to `Box<Integer>`; we have to take the developer at their word that the specified `box` really is a `Box<Integer>`. But the heap pollution is not caught right away; only when we try to *use* the `String` that was in the box as an `Integer`, do we detect that something went wrong. Under the translation we have, if we cast our box to `Box<Integer>` and back to `Box<String>` before we used it as a `Box<String>`, nothing bad happens (for better or worse.) Under a heterogeneous translation, `Box<String>` and `Box<Integer>` would have different runtime types, and this cast would fail.

The language actually provides quite a strong safety guarantee for generics, as long as we follow the rules:

> If a program compiles with no unchecked or raw warnings, the synthetic casts inserted by the compiler will never fail.

In other words, *heap pollution can only occur when we are interoperating with non-generic code* or when we *lie to the compiler*. At the point where the heap pollution is discovered, we get a clean exception telling us what type was expected and what type was actually found.

## Context: Ecosystem of JVM implementations and languages

The design choices surrounding generics were also influenced by the structure of the ecosystem of JVM implementations and of languages running on the JVM. While to most developers "Java" is a monolithic entity, in fact the Java Language and the Java Virtual Machine (JVM) are separate entities, each with their own specification. The Java compiler produces classfiles for the JVM (whose format and semantics are laid out in the Java Virtual Machine Specification), but the JVM will happy run any valid classfile, regardless of what source language it originally came from. By some counts, there are over 200 languages that use the JVM as compilation target, some of which have a lot in common with the Java language (e.g., Scala, Kotlin) and others which are very different languages (e.g., JRuby, Jython, Jaskell.)

One reason the JVM has been so successful as a compilation target, even for languages quite different from Java, is that it provides a fairly abstract model for computing, with limited influence from the Java language. The abstraction layer between the language and virtual machine was not only useful for stimulating an ecosystem of other languages that run on the JVM, but also an ecosystem of *independent implementations* of the JVM. While the market today has consolidated substantially, at the time that generics were added to Java, there were over a dozen commercially viable implementations of the JVM. Reifying generics would mean that not only would we need to enhance the language to support generics, but also the JVM.

While it might have been technically possible to add generic support to the JVM at the time, not only would it have been a significant engineering investment requiring substantial coordination and agreement between the many implementors, the ecosystem of languages on the JVM might also have had an opinion about reified generics. If, for example, the interpretation of reification included type checking at runtime, would Scala (with its declaration-site variance) be happy to have the JVM enforce Java's (invariant) generic subtyping rules?

## Erasure was the pragmatic compromise

Taken together, these constraints (both technical and ecosystem) acted as a powerful force to push us towards a homogeneous translation strategy where generic type information is erased at compile time. To summarize, forces that pushed us towards this decision include:

- **Runtime costs.** A heterogeneous translation entails all sorts of runtime costs: greater static and dynamic footprint, greater class-loading costs, greater JIT costs and code cache pressure, etc. This might have put developers in a position where they had to choose between type-safety and performance.

- **Migration compatibility.** There was no known translation scheme at the time that would have allowed a migration to reified generics to be source- and binary-compatible, creating flag days and invalidating developer's considerable investment in their existing code.

- **Runtime costs, bonus edition.** If reification is interpreted as *checking* types at runtime (just as stores into Java's covariant arrays are dynamically checked), this would have a significant runtime impact, as the JVM would have to perform generic subtyping checks at runtime on every field or array element store, using the language's generic type system. (This might sound easy and cheap when the type is something simple like `List<String>`, but can quickly get expensive when they are something like `Map<? extends List<? super Foo>>, ? super Set<? extends Bar>>`. (In fact, later research cast doubt on the decidability of generic subtyping).

- **JVM ecosystem.** Getting a dozen JVM vendors to agree on if, and how, type information would be reified at runtime was a highly questionable proposition.

- **Delivery pragmatics.** Even if it were possible to get a dozen JVM vendors to agree on a scheme that could actually work, it would have greatly increased the complexity, timeline, and risk of an already substantial and risky effort.

- **Language ecosystem.** Languages like Scala might not have been happy to have Java's invariant generics burned into the semantics of the JVM. Agreeing on a set of acceptable cross-language semantics for generics in the JVM would again have increased the complexity, timetable, and risk of an already substantial and risky effort.

- **Users would have to deal with erasure (and therefore heap pollution) anyway.** Even if type information could be preserved at runtime, there would always be dusty classfiles that were compiled before the class was generified, so there would still be the possibility that any given `ArrayList` in the heap had no type information attached, with the attendant risk of heap pollution.

- **Certain useful idioms would have been inexpressible.** Existing generic code will occasionally resort to unchecked casts when it knows something about runtime types that the compiler does not, and there is no easy way to express it in the generic type system; many of these techniques would have been impossible with reified generics, meaning that they would have to have been expressed in a different, and often far more expensive, way.

It is clear that the costs and risks would have been substantial; what would have been the benefits? Earlier, we cited three possible benefits of reification: reflection, layout specialization, and run-time type checking. The above arguments largely rule out the possibility we would have gotten run time type checking (runtime cost, undecidability risk, ecosystem risk, and the existence of erased instances).

Surely it would be nice to be able to ask a `List` what its element type is (and maybe it could have answered, but maybe not) — this is clearly of nonzero benefit. It is just that the costs and benefits were out of line by several orders of magnitude. (Another cost of the chosen translation strategy is that primitives can not be supported as type parameters; instead of `List<int>`, we have to use `List<Integer>`.)

The common misconception that erasure is "a dirty hack" generally stems from a lack of awareness of what the true costs of the alternative would have been, both in engineering effort, time to market, delivery risk, performance, ecosystem impact, and programmer convenience given the large volume of Java code already written and the diverse ecosystem of both JVM implementations and languages running on the JVM.

© 2026 Oracle Corporation and/or its affiliates
Terms of Use · License: GPLv2 · Privacy · Trademarks
fda9b9069182 · 2021/12/17 22:57 UTC